# Relatório de Práticas: REST API com Spring Boot

**Aluno:** Victor Rodrigues Cavalcante Rocha
**Turma:** B - Análise e Desenvolvimento de Sistemas (ADS) - 2º Semestre
**Disciplina:** Back-End

Este relatório documenta o Laboratório Guiado **StoreAPI**: uma API REST de produtos construída com Spring Boot, Spring Data JPA e banco H2 (Java 21), testada pelo Postman.

## Prática 1: Preparar e executar o projeto

O projeto `storeapi-inicial` foi aberto na IDE e executado com Java 21 e Maven (`mvn spring-boot:run`). A API fica disponível em `http://localhost:8080/api/produtos` e o console do H2 em `/h2-console` (JDBC URL `jdbc:h2:mem:storedb`, usuário `sa`, senha vazia). A tabela **PRODUTO** é criada automaticamente pela propriedade `spring.jpa.hibernate.ddl-auto` do `application.properties`, que gera o esquema do banco a partir das entidade

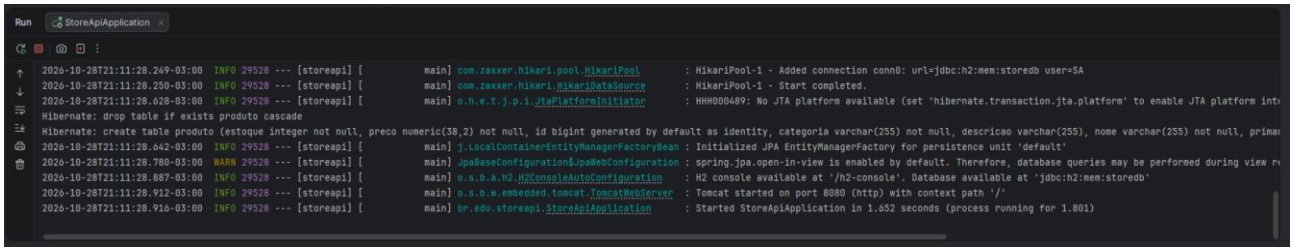



*Figura 1: Aplicação Spring Boot em execução no console da IDE.*

## Prática 2: Entidade e persistência

A classe `Produto` usa `@Entity` para indicar que representa uma tabela. `@Id` marca a chave primária, `@GeneratedValue` faz o banco gerar o id automaticamente e `@Column` configura as colunas. Cada atributo da classe vira uma coluna e cada objeto vira uma linha da tabela **PRODUTO**: é o mapeamento objeto-relacional (ORM) feito pelo JPA/Hibernate.

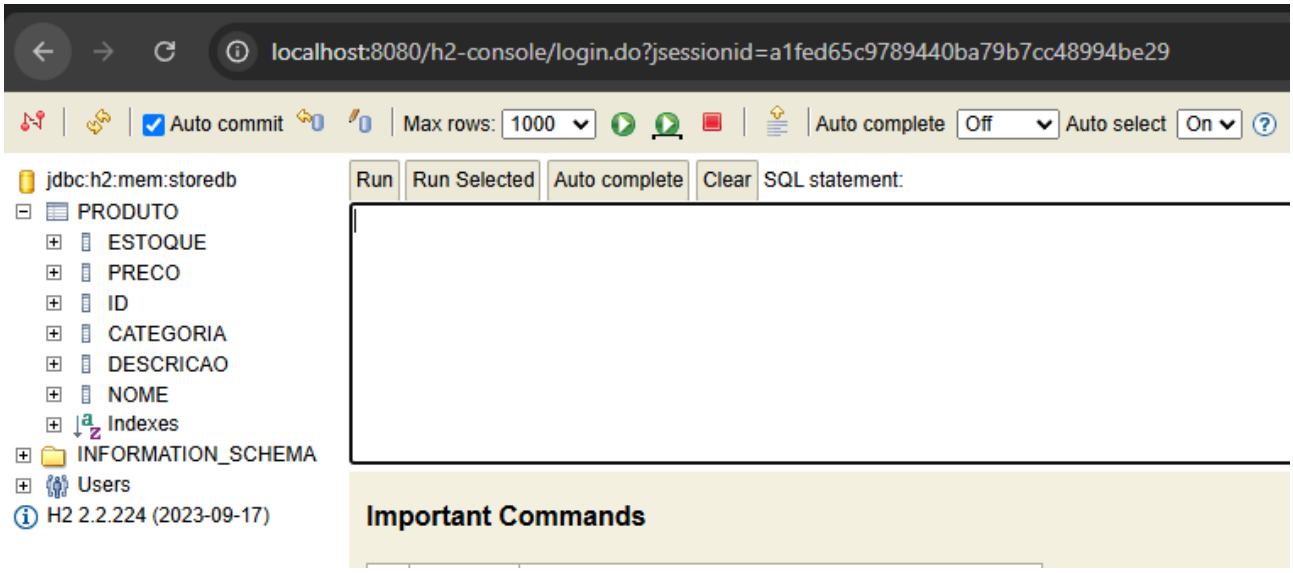


*Figura 2: Tabela PRODUTO e suas colunas no console do H2.*

## Prática 3: Repository

O `ProdutoRepository` estende `JpaRepository<Produto, Long>`, que já traz os métodos de CRUD (`save`, `findById`, `findAll`, `existsById`, `deleteById`). Por isso não foi necessário escrever SQL: o Spring Data JPA gera a implementação em tempo de execução. O método `findByNomeContainingIgnoreCase` também é gerado automaticamente a partir do próprio nome do método.

## Prática 4: GET - listagem

No `ProdutoController`, `@RestController` define que os retornos são escritos como JSON, `@RequestMapping("/api/produtos")` define a rota base e `@GetMapping` mapeia a listagem. Antes de cadastrar qualquer registro, o esperado é a resposta `[]` com status 200.

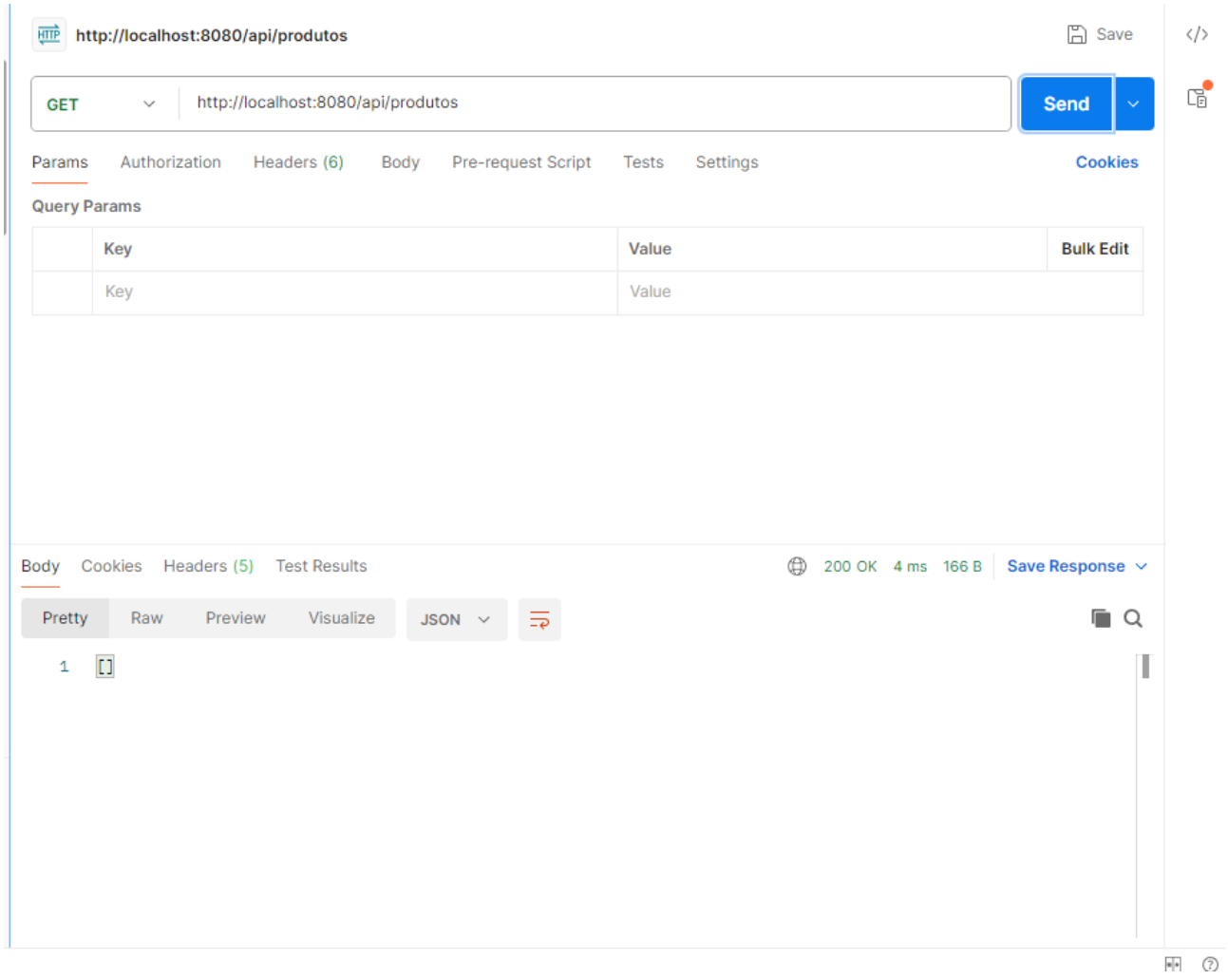



*Figura 3: GET /api/produtos antes de cadastrar registros.*

## Prática 5: POST - cadastro

Foi enviado um `POST` para `/api/produtos` com `Content-Type: application/json` e o corpo do produto "Notebook Dell". A API respondeu **201 Created** e devolveu o produto com o `id` gerado pelo banco (id 1). Outros produtos foram cadastrados em seguida para os testes das próximas práticas.

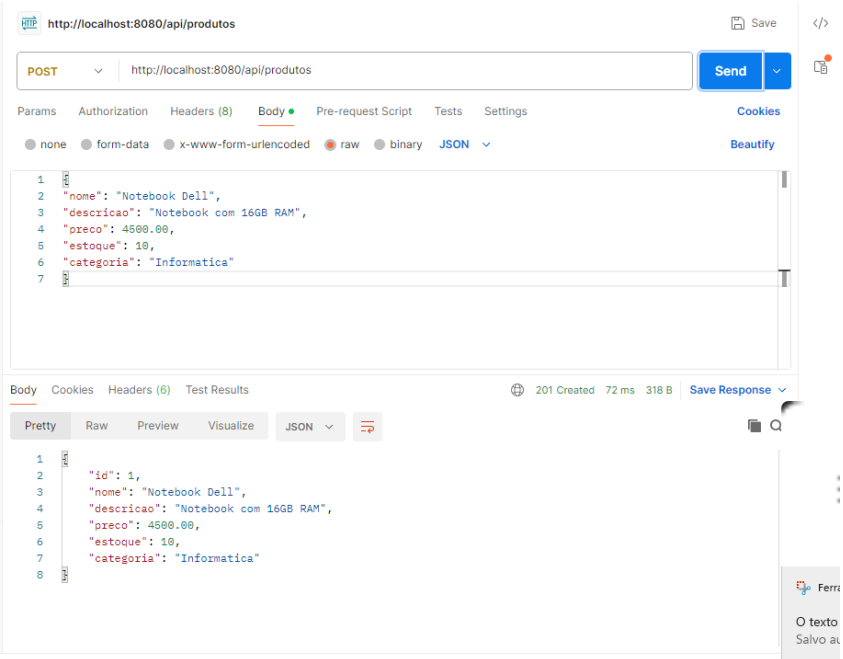



*Figura 4: POST /api/produtos no Postman: produto cadastrado, id 1 gerado e status 201 Created.*

# Prática 6: GET por ID

Foram comparadas as duas respostas possíveis da rota `GET /api/produtos/{id}`. No código, `@PathVariable` recebe o id da URL, `findById` devolve um `Optional` e o `ResponseEntity` permite escolher o status: **200 OK** com o JSON quando o produto existe e **404 Not Found** sem corpo quando não existe.

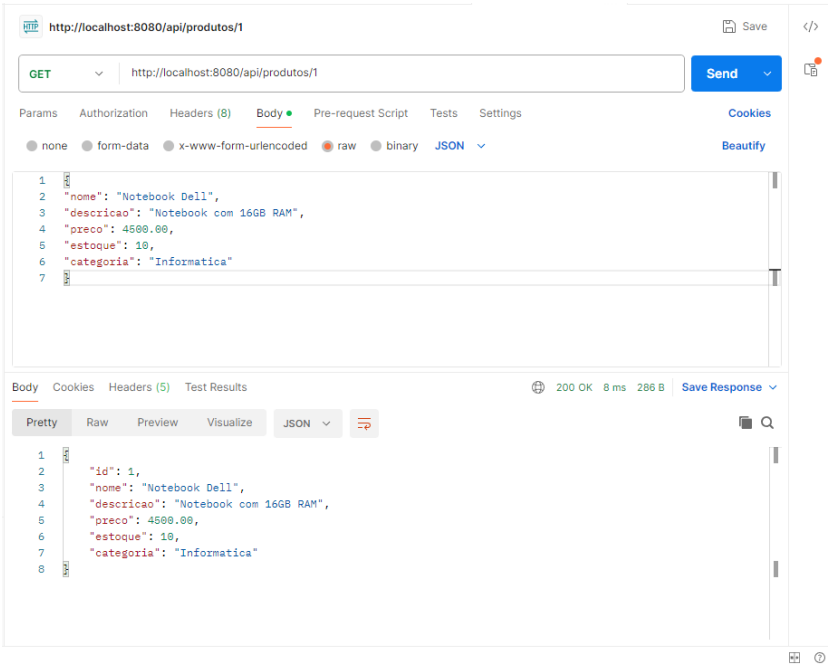



*Figura 5: GET /api/produtos/1: ID existente retorna 200 OK com os dados do produto.*

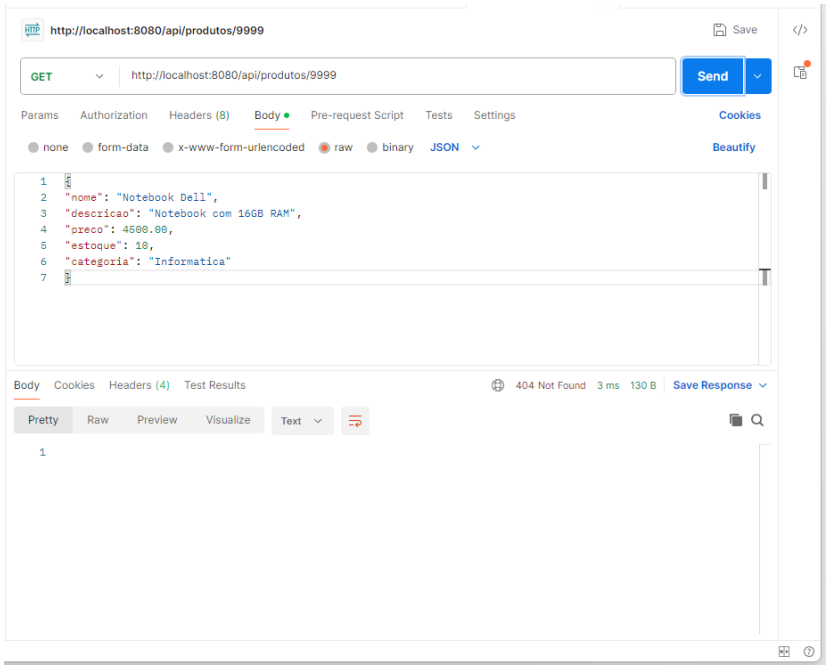



*Figura 6: GET /api/produtos/9999: ID inexistente retorna 404 Not Found, sem corpo.*

# Prática 7: Atualizar (PUT) e Excluir (DELETE)

O objetivo desta prática era implementar o método HTTP PUT para atualizar os dados de um produto existente na base de dados H2, utilizando a anotação `@PutMapping` no controlador, e o DELETE com `@DeleteMapping`. No PUT, o registro é buscado pelo id, seus atributos são atualizados com os dados do `@RequestBody` e persistidos com `save()`. No DELETE, verifica-se com `existsById` se o produto existe e, em caso positivo, ele é removido com `deleteById()`.

```
// Prática 7: Atualizar (PUT /api/produtos/{id})
@PutMapping("/{id}")
public ResponseEntity<Produto> atualizar(@PathVariable Long
 return repository.findById(id)
         .map(produtoExistente -> {
          // Atualiza os campos do produto existente com os
          produtoExistente.setNome(dto.nome());
          produtoExistente.setDescricao(dto.descricao());
          produtoExistente.setPreco(dto.preco());
          produtoExistente.setEstoque(dto.estoque());
          produtoExistente.setCategoria(dto.categoria());

          // Salva as alterações
          Produto atualizado = repository.save(produtoExiste
          return ResponseEntity.ok(atualizado);
         })
         .orElseGet(() -> ResponseEntity.notFound().build())
}

// Prática 7: Excluir (DELETE /api/produtos/{id})
@DeleteMapping("/{id}")
public ResponseEntity<Void> excluir(@PathVariable Long id) {
 if (!repository.existsById(id)) {
  return ResponseEntity.notFound().build();
```

*Figura 7: Implementação dos métodos atualizar (PUT) e excluir (DELETE) no ProdutoController.*

> **Resultado Esperado:** Ao enviar um JSON com dados alterados para um ID existente, a API deve processar a alteração e retornar o Status **200 OK** com os dados atualizados. No DELETE, o esperado é **204 No Content** (ou 404 se o ID não existir).

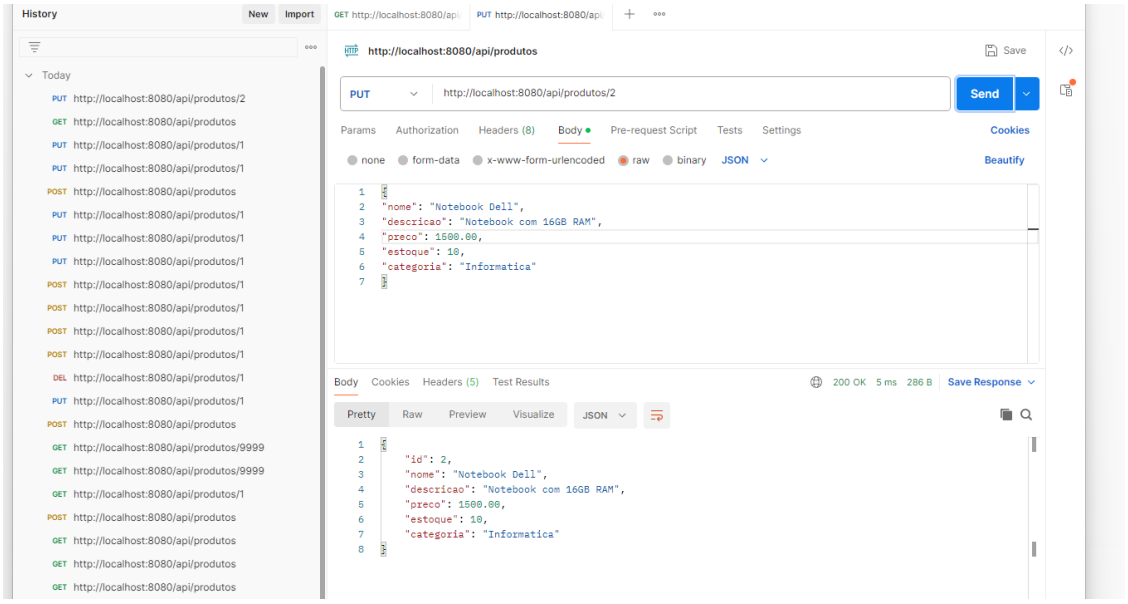



*Figura 8: Teste bem-sucedido no Postman. Produto ID 2 atualizado com o novo preço (1500.00), retornando 200 OK.*

# Prática 8: Pesquisa por nome

A rota `GET /api/produtos/busca?nome=note` recebe o termo pela query string com `@RequestParam` e chama o método `findByNomeContainingIgnoreCase`. A pesquisa não diferencia maiúsculas de minúsculas por causa do *IgnoreCase* no nome do método, e o *Containing* faz a busca por trecho do nome (equivalente a um LIKE). Assim, "note" encontra "Notebook Dell".

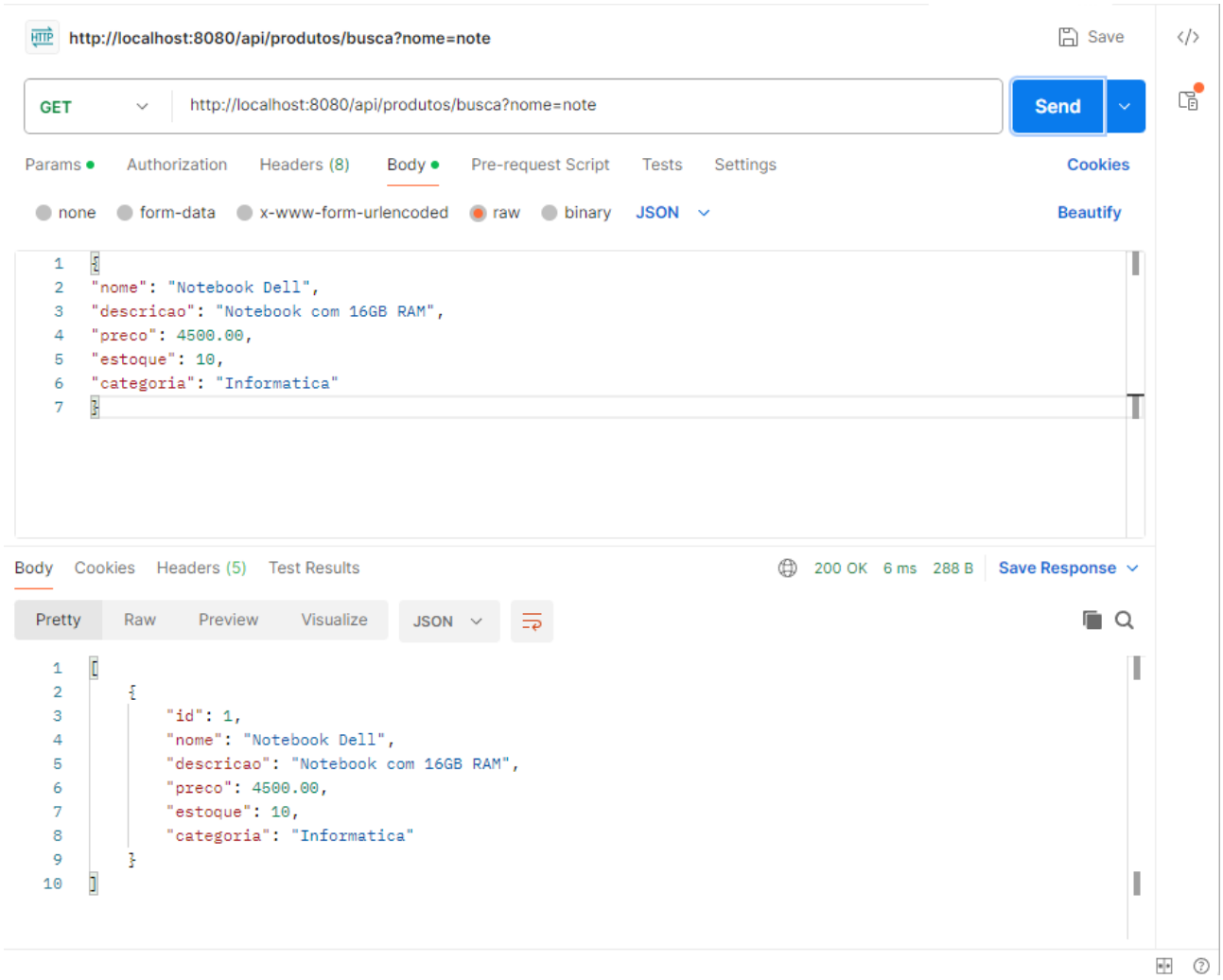



*Figura 10: GET /api/produtos/busca?nome=note.*

# Troubleshooting e Resolução de Erros

Durante o desenvolvimento e testes das rotas, foram encontrados e resolvidos alguns erros HTTP comuns, o que ajudou a compreender melhor o funcionamento do protocolo e da persistência em memória (H2).

> **Análise do Erro 404 (Not Found):** Ao testar o método PUT no ID 1, a API retornou 404. Isto validou a lógica do código, pois o produto ID 1 havia sido apagado anteriormente no teste do método DELETE.

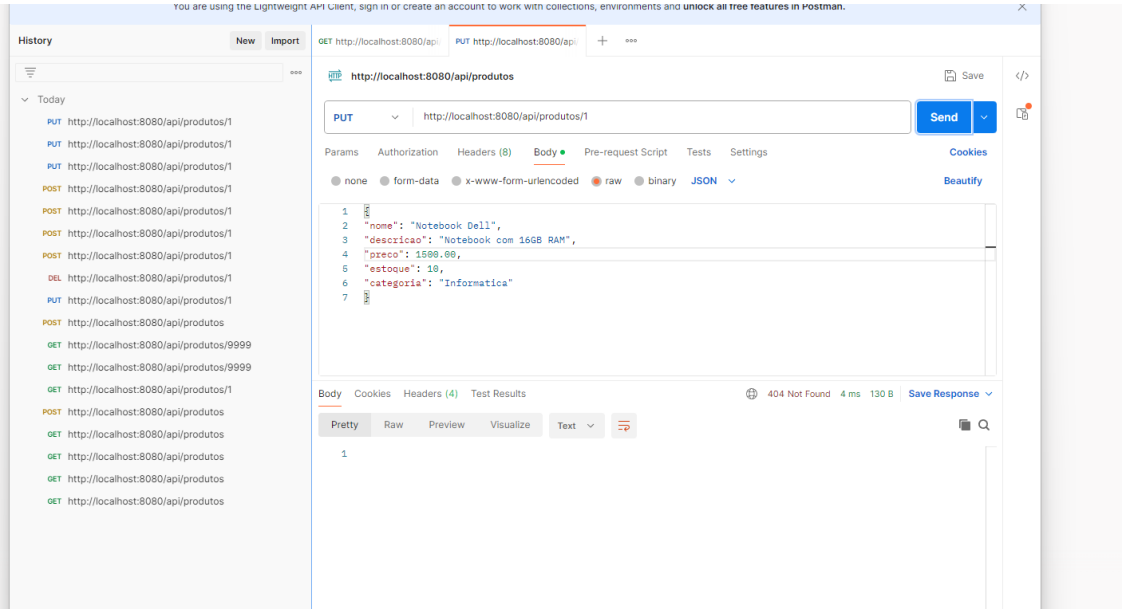



*Figura 11: Comportamento correto da API retornando 404 ao tentar atualizar um ID inexistente.*

### Incompatibilidade de Tipos (DTO vs Entity)

Na transição para a Prática 9, ocorreu um erro de compilação devido à divergência de tipagem forte no Java. O atributo `preco` estava como `Double` no Record (DTO), mas a Entidade esperava `BigDecimal`.

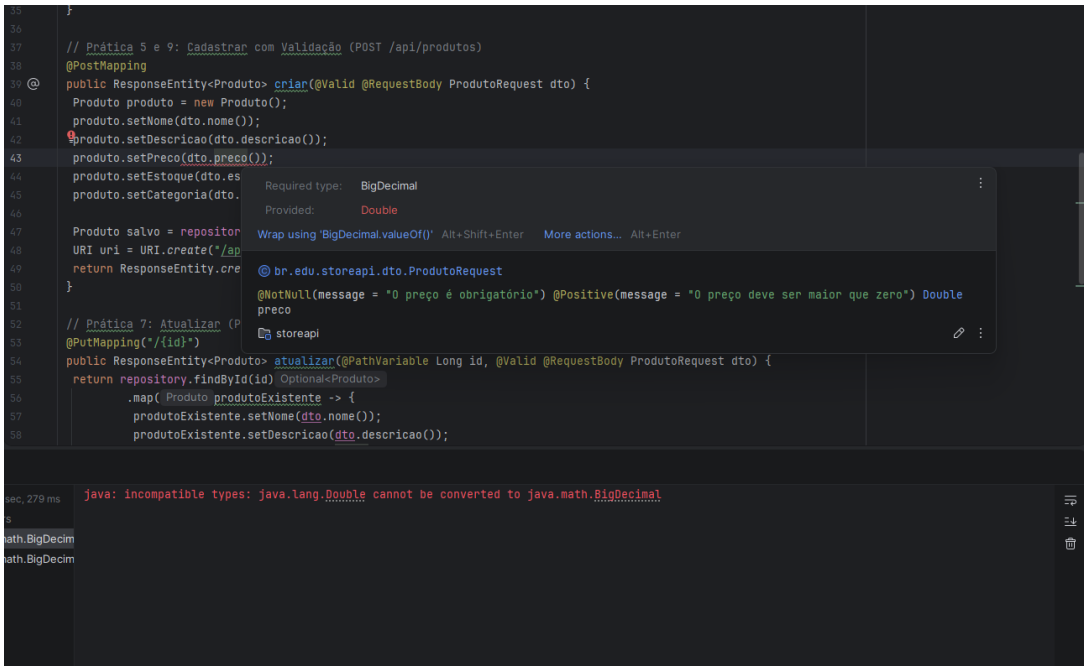



*Figura 12: Identificação do erro de tipagem no IntelliJ, posteriormente corrigido alinhando ambos para BigDecimal.*

# Prática 9: Validação de Dados (POST)

Implementação do Jakarta Validation (`@Valid`, `@Positive`, `@NotBlank`) no DTO para garantir a integridade dos dados antes de chegarem à base de dados.

> **Resultado Esperado:** Ao tentar criar um produto com um valor inválido (ex: preço negativo), a API deve bloquear a requisição e retornar o Status **400 Bad Request**.

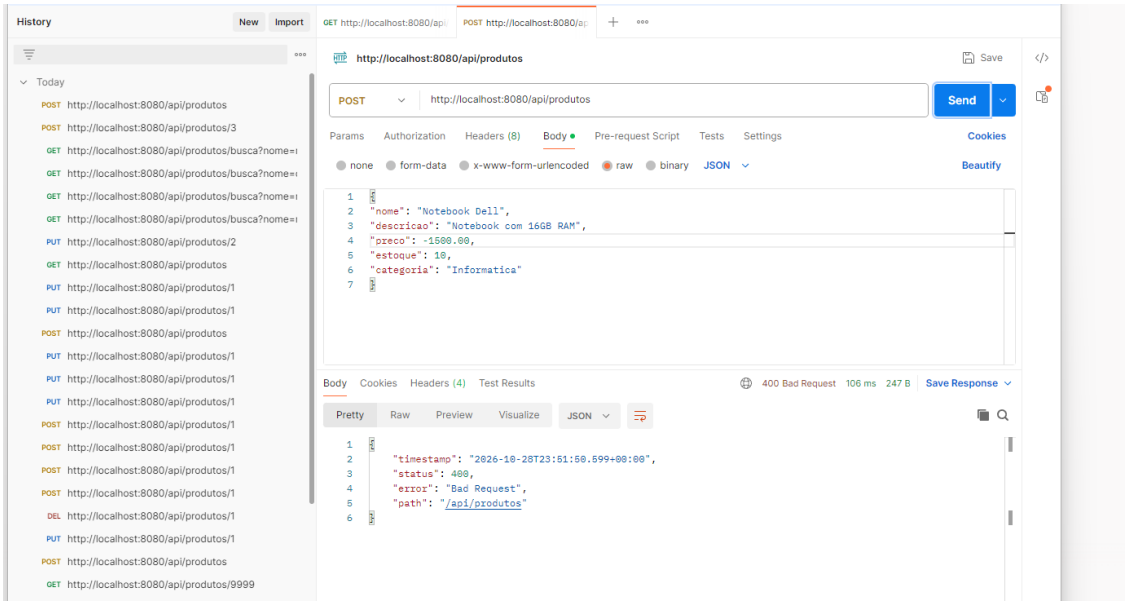



*Figura 13: Sucesso na validação. O Spring Boot interceptou o preço negativo (-1500.00) e retornou 400 Bad Request.*

# Prática 10: Checklist de testes

| Teste | Status esperado | Evidência |
|---|---|---|
| GET lista retorna JSON | 200 OK | Figura 3 (pendente) |
| POST cadastra | 201 Created | Figura 4 |
| GET ID existente | 200 OK | Figura 5 |
| GET ID inexistente | 404 Not Found | Figura 6 |
| PUT atualiza | 200 OK | Figura 8 |
| PUT com ID inexistente | 404 Not Found | Figura 11 |
| DELETE exclui | 204 No Content | Figura 9 (pendente) |
| Pesquisa por nome | 200 OK | Figura 10 (pendente) |
| POST inválido (DTO) | 400 Bad Request | Figura 13 |
| Registros visíveis no H2 | - | Figura 2 (pendente) |

# Documentação e Entrega

Para concluir o laboratório, o arquivo `README.md` foi atualizado com as respostas às questões teóricas propostas (@Controller vs @RestController, @RequestBody, @PathVariable vs @RequestParam, ResponseEntity e códigos HTTP 200, 201, 204 e 404), formatado corretamente em Markdown.

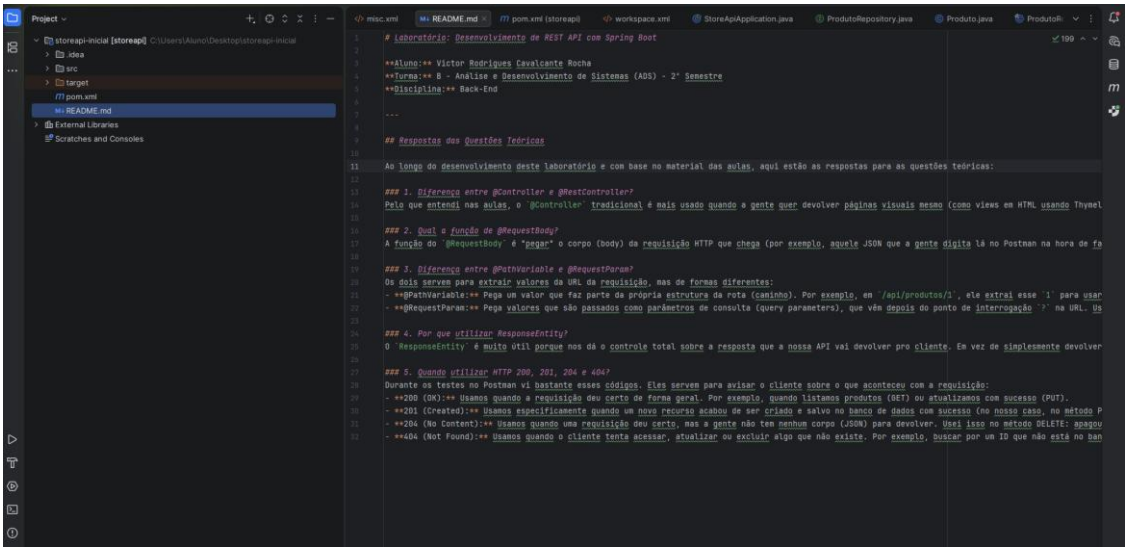


*Figura 14: Arquivo README finalizado com as justificativas teóricas sobre anotações e códigos HTTP.*